

Namespace Hse.EviDense

Classes

[AdcResult](#)

Represents the ADC values.

[Channel](#)

Represents a measurement channel with sample and reference values, measured in uV.

[Device](#)

This class represents the eviDense module.

[Dict](#)

Defines constant dictionary keys used in measurement processing.

[Factors](#)

Correction factors due to different light absorption at different wavelengths of the cuvette.

[LevellingChannelResult](#)

Represents the result of a levelling operation for a single measurement channel.

[LevellingResult](#)

Represents the overall levelling results for multiple measurement channels.

[Measurement](#)

Represents a measurement containing baseline, air, and sample values, along with optional comments.

[Quadruple](#)

Represents a quadruple set of measurement values at different wavelengths. This class performs mathematical operations (plus, minus, ...) on all four wavelengths at the same time.

[Results](#)

Represents the results of a nucleic acid analysis, including DNA and RNA concentrations and purity ratios.

[SelfTestResult](#)

Represents the result of a self-test, containing various checks and problem indicators.

[SingleMeasurement](#)

Represents a single measurement consisting of multiple wavelength channels.

[StorageMeasurement](#)

Handles storage and retrieval of measurements using JSON serialization.

[USB](#)

Provides USB-related constants.

Enums

[Amplifier](#)

Enumeration representing the two amplifiers.

[AmplifierFactors](#)

Enumeration representing amplification factor of the amplifiers.

[Error](#)

Enumeration representing various error codes.

[Index](#)

Enumeration representing different index values for hardware configurations.

[Led](#)

Enumeration representing different led wave lengths.

[Selftest](#)

Flags enumeration for self-test components.

[StatusLedColor](#)

Enumeration representing different status led colors .

[TypeOf](#)

Enumeration representing different data types.

Class AdcResult

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents the ADC values.

```
public class AdcResult
```

Inheritance

[object](#)  ← AdcResult

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

AdcResult(int, int, int, int)

```
public AdcResult(int sampleValue, int referenceValue, int vref, int current)
```

Parameters

sampleValue [int](#) 

referenceValue [int](#) 

vref [int](#) 

current [int](#) 

Properties

Current

LED current in uA.

```
public int Current { get; set; }
```

Property Value

[int](#)

ReferenceValue

Voltage of reference detector in uV.

```
public int ReferenceValue { get; set; }
```

Property Value

[int](#)

SampleValue

Voltage of sample detector in uV.

```
public int SampleValue { get; set; }
```

Property Value

[int](#)

Vref

Voltage of internal 5V reference in uV.

```
public int Vref { get; set; }
```

Property Value

[int](#)

Enum Amplifier

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing the two amplifiers.

```
public enum Amplifier
```

Fields

```
REFERENCE = 1
```

```
SAMPLE = 0
```

Enum AmplifierFactors

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing amplification factor of the amplifiers.

```
public enum AmplifierFactors
```

Fields

```
F_111_0 = 2
```

```
F_11_0 = 1
```

```
F_1_1 = 0
```

Class Channel

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents a measurement channel with sample and reference values, measured in uV.

```
public class Channel
```

Inheritance

[object](#)  ← Channel

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

Channel(double, double)

Initializes a new instance of the [Channel](#) class with optional sample and reference values.

```
public Channel(double sample = 0, double reference = 0)
```

Parameters

sample [double](#) 

The sample value in uV (default: 0.0).

reference [double](#) 

The reference value in uV (default: 0.0).

Properties

Reference

Gets or sets the reference value in uV.

```
public double Reference { get; set; }
```

Property Value

[double](#)[↗]

Sample

Gets or sets the sample value in uV.

```
public double Sample { get; set; }
```

Property Value

[double](#)[↗]

Methods

FromJson(JsonNode)

Creates a [Channel](#) instance from a JSON node.

```
public static Channel FromJson(JsonNode node)
```

Parameters

node [JsonNode](#)[↗]

The JSON node containing the channel data.

Returns

[Channel](#)

A new [Channel](#) instance with values extracted from the JSON node.

Exceptions

[ArgumentException](#) 

Thrown when the JSON node is invalid.

ToJson()

Converts the channel values to a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#) 

A JsonNode representing the channel values.

ToString()

Returns a string representation of the channel values.

```
public override string ToString()
```

Returns

[string](#) 

A string in the format "Sample: [value] Reference: [value]".

Class Device

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

This class represents the eviDense module.

```
public class Device : IDisposable
```

Inheritance

[object](#)  ← Device

Implements

[IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

Device(string?)

Initializes a new instance of the [Device](#) class. The constructor attempts to connect to an EviDense module via a serial port. If no serial port is provided, it tries to auto-detect the device.

Throws an exception if:

- No device is found during auto-detection.
- The provided serial port does not match the expected VID/PID for the EviDense device.

```
public Device(string? serialNumber = null)
```

Parameters

serialNumber [string](#) 

The serial number of the device to connect to. If null or empty, the constructor attempts to automatically find a device.

Exceptions

[Exception](#)

Thrown when:

- No device is found during auto-detection (if `serialPortName` is null or empty).
- The provided serial port does not correspond to an EviDense device.

Properties

LibraryVersion

```
public string LibraryVersion { get; }
```

Property Value

[string](#)

Methods

AddSelfTestDetails(JsonNode?)

Adds detailed self-test measurement data to a given JSON node. The method retrieves various self-test parameters from the device and stores them in the provided JSON structure, including amplifier values and LED measurement data for different wavelengths.

```
public void AddSelfTestDetails(JsonNode? node)
```

Parameters

`node` [JsonNode](#)

A [JsonNode](#) object where the self-test details will be stored.

Remarks

The method follows these steps:

1. Retrieves amplifier-related self-test values and adds them to the JSON node.
2. Uses the helper method `LedAsJson` to fetch and structure LED-specific measurement data for different wavelengths (230nm, 260nm, 280nm, and 340nm).
3. Each LED entry contains:
 - ILED (LED intensity)
 - Dark Sample
 - Dark Reference
 - Sample
 - Reference

Exceptions

[ArgumentNullException](#)↗

Thrown if the provided `node` is `null`.

Baseline()

Retrieves the baseline measurement data from the device and constructs a [SingleMeasurement](#) object.

```
public SingleMeasurement Baseline()
```

Returns

[SingleMeasurement](#)

A [SingleMeasurement](#) object containing the baseline data for the four channels (230nm, 260nm, 280nm, and 340nm).

Remarks

This command expects an empty cuvette guide. If required, the module executes a [Levelling\(\)](#) command internally and then starts a measurement. if needed and starts after wards a measurement.

Exceptions

[FormatException](#)↗

Thrown if any of the response values from the device cannot be parsed as integers.

[Exception](#)↗

Thrown if the device response is invalid or does not contain the expected number of values.

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

~Device()

```
protected ~Device()
```

FirmwareVersion()

Retrieves the firmware version of the connected device. This method uses the [Get<T>\(Index\)](#) method to fetch the firmware version from the device at the predefined index for firmware versions.

```
public string FirmwareVersion()
```

Returns

[string](#) 

The firmware version of the device as a string.

Exceptions

[InvalidCastException](#) 

Thrown if the retrieval or conversion of the firmware version fails.

[Exception](#) 

Thrown if there is an issue communicating with the device or retrieving the value.

FwUpdate(string)

Performs a firmware update on the device. The method reads the firmware file, erases the existing firmware, writes the new firmware line by line, and then verifies its validity. If the update fails at any stage, an exception is thrown.

```
public void FwUpdate(string filename)
```

Parameters

filename [string](#) 

The path to the firmware file that contains the new firmware image.

Remarks

The update process follows these steps:

1. Read all lines from the specified firmware file.
2. Erase the existing firmware using `Hse.EviDense.Device.erase()`.
3. Send each line of the new firmware to the device using the "S {line}" command.
4. Verify the firmware using [verify\(\)](#); if verification fails, throw an exception.
5. Reboot the device and close the serial port.
6. Wait for 30 seconds to allow the device to restart.
7. Reopen the serial port and clear its buffers.
8. Verify the firmware again; if verification still passes, throw an exception indicating the update failed.

Exceptions

[IOException](#) 

Thrown if the firmware file cannot be read.

[Exception](#) 

Thrown if the firmware update fails at any stage (e.g., verification fails before or after reboot).

GetAdcValues()

Get ADC values.

```
public AdcResult GetAdcValues()
```

Returns

[AdcResult](#)

GetAvailableDevices()

Retrieves a list of available eviDense devices by scanning serial ports.

```
public static List<string> GetAvailableDevices()
```

Returns

[List](#) <[string](#)>

A list of serial numbers of available matching devices.

Get<T>(Index)

Retrieves a value from the device at the specified index and converts it to the desired type. Sends a command to the device to fetch the value and parses the response.

```
public T Get<T>(Index index)
```

Parameters

index [Index](#)

The index from which the value should be retrieved.

Returns

T

The value retrieved from the device, converted to the specified type T.

Type Parameters

T

The type to which the retrieved value will be converted. Must be compatible with [ChangeType\(object, Type\)](#).

Exceptions

[InvalidCastException](#)

Thrown if the conversion to the specified type `T` fails.

[Exception](#)

Thrown if the device response is invalid or does not contain the expected data format.

IsCuvetteHolderEmpty()

Checks whether the cuvette holder is empty.

```
public bool IsCuvetteHolderEmpty()
```

Returns

[bool](#)

`true` if the cuvette holder is empty; otherwise, `false`.

Exceptions

[FormatException](#)

Thrown if the response value from the device cannot be parsed as an integer.

[IndexOutOfRangeException](#)

Thrown if the device response does not contain the expected number of values.

Levelling()

Retrieves levelling data from the device and parses it into a [LevellingResult](#) object.

```
public LevellingResult Levelling()
```


Returns

[LevellingResult](#)

A [LevellingResult](#) object containing the parsed levelling data for all channels.

Remarks

This command expects an empty cuvette guide. Each LED channel current is set so that the detector measures 90% of the maximum level.

Exceptions

[FormatException](#)

Thrown if any of the device's response values cannot be parsed as integers.

[Exception](#)

Thrown if the device response is invalid or does not contain the expected number of values.

Measure()

Performs a measurement and retrieves data from the device.

```
public SingleMeasurement Measure()
```

Returns

[SingleMeasurement](#)

A [SingleMeasurement](#) object containing the measured data for four channels.

Exceptions

[FormatException](#)

Thrown if any of the response values from the device cannot be parsed as integers.

[IndexOutOfRangeException](#)

Thrown if the device response does not contain the expected number of values.

ProductionNumber()

Retrieves the production number of the connected device. This method uses the [Get<T>\(Index\)](#) method to fetch the production number from the device at the predefined index.

```
public string ProductionNumber()
```

Returns

[string](#) 

The production number of the device as a string.

Exceptions

[Exception](#) 

Thrown if there is an issue communicating with the device or retrieving the value.

SelfTest()

Performs a self-test on the device and retrieves the result.

```
public SelfTestResult SelfTest()
```

Returns

[SelfTestResult](#)

A [SelfTestResult](#) object containing the parsed self-test result.

Exceptions

[FormatException](#) 

Thrown if the response value from the device cannot be parsed as an integer.

[IndexOutOfRangeException](#) 

Thrown if the device response does not contain the expected number of values.

SerialNumber()

Retrieves the serial number of the connected device. This method uses the [Get<T>\(Index\)](#) method to fetch the serial number from the device at the predefined index for serial numbers.

```
public string SerialNumber()
```

Returns

[string](#) 

The serial number of the device as a string.

Exceptions

[InvalidCastException](#) 

Thrown if the retrieval or conversion of the serial number fails.

[Exception](#) 

Thrown if there is an issue communicating with the device or retrieving the value.

Set(Index, double)

Sets a value on the device at the specified index. Sends a command to update the value on the device.

```
public void Set(Index index, double value)
```

Parameters

index [Index](#)

The index where the value should be set.

value [double](#) 

The double value to set at the specified index.

Set(Index, int)

Sets a value on the device at the specified index. Sends a command to update the value on the device.

```
public void Set(Index index, int value)
```

Parameters

index [Index](#)

The index where the value should be set.

value [int](#)

The integer value to set at the specified index.

Set(Index, string)

Sets a value on the device at the specified index. Sends a command to update the value on the device.

```
public void Set(Index index, string value)
```

Parameters

index [Index](#)

The index where the value should be set.

value [string](#)

The string value to set at the specified index.

SetAmplifiyingFactor(Amplifier, AmplifierFactors)

Set the amplification factor of the given amplifier.

```
public void SetAmplifiyingFactor(Amplifier amplifier, AmplifierFactors factor)
```

Parameters

amplifier [Amplifier](#)

factor [AmplifierFactors](#)

SetLedAndCurrent(Led, int)

Sets the current of the given LED. Current is in uA.

```
public void SetLedAndCurrent(Led led, int current)
```

Parameters

led [Led](#)

current [int](#)[↗](#)

SetStatusLed(StatusLedColor)

Sets the status led color.

```
public void SetStatusLed(StatusLedColor color)
```

Parameters

color [StatusLedColor](#)

TechnicalReport()

Generates a technical report containing various diagnostic data from the device. The report includes levelling results, self-test data, serial number, and firmware version.

```
public JsonNode TechnicalReport()
```

Returns

[JsonNode](#)[↗](#)

A [JsonNode](#)[↗](#) object containing structured diagnostic data.

Remarks

The generated report consists of the following fields:

- [LEVELLING](#): The levelling results in JSON format.
- [SELFTEST](#): The base self-test results in JSON format, with additional details added via [AddSelfTestDetails\(JsonNode?\)](#).
- [SERIALNUMBER](#): The device's serial number.
- [FIRMWAREVERSION](#): The firmware version currently installed on the device.
- [PRODUCTIONNUMBER](#): The device's production number.

ToString()

Returns a string representation of the device, including its serial port, serial number, and firmware version.

```
public override string ToString()
```

Returns

[string](#)

A formatted string containing the device's port name, serial number, and firmware version.

Remarks

The returned string follows the format: "eviDense Module@{PortName} SN:{SerialNumber} Version: {FirmwareVersion}" Example output: "eviDense Module@COM3 SN:12345678 Version:1.0.5"

verify()

Verifies if the 2nd firmware image is valid.

```
public bool verify()
```

Returns

[bool](#)

true if the verification was successful; otherwise, **false**.

Exceptions

[FormatException](#)

Thrown if the response value from the device cannot be parsed as an integer.

[IndexOutOfRangeException](#)

Thrown if the device response does not contain the expected number of values.

Class Dict

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Defines constant dictionary keys used in measurement processing.

```
public class Dict
```

Inheritance

[object](#)  ← Dict

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

AIR

```
public const string AIR = "air"
```

Field Value

[string](#) 

BASELINE

```
public const string BASELINE = "baseline"
```

Field Value

[string](#) 

CH_230

```
public const string CH_230 = "230"
```

Field Value

[string](#)

CH_260

```
public const string CH_260 = "260"
```

Field Value

[string](#)

CH_280

```
public const string CH_280 = "280"
```

Field Value

[string](#)

CH_340

```
public const string CH_340 = "340"
```

Field Value

[string](#)

COMMENT

```
public const string COMMENT = "comment"
```

Field Value

[string](#) 

FIRMWAREVERSION

```
public const string FIRMWAREVERSION = "firmwareVersion"
```

Field Value

[string](#) 

LEVELLING

```
public const string LEVELLING = "levelling"
```

Field Value

[string](#) 

LEVELLING_AMPLIFIERREFERENCE

```
public const string LEVELLING_AMPLIFIERREFERENCE = "amplifierreference"
```

Field Value

[string](#) 

LEVELLING_AMPLIFIERSAMPLE

```
public const string LEVELLING_AMPLIFIERSAMPLE = "amplifiersample"
```

Field Value

[string](#) 

LEVELLING_CURRENT

```
public const string LEVELLING_CURRENT = "current"
```

Field Value

[string](#) 

LEVELLING_RESULT

```
public const string LEVELLING_RESULT = "result"
```

Field Value

[string](#) 

MEASUREMENTS

```
public const string MEASUREMENTS = "measurements"
```

Field Value

[string](#) 

PRODUCTIONNUMBER

```
public const string PRODUCTIONNUMBER = "productionnumber"
```

Field Value

[string](#)

PURITY260_230

```
public const string PURITY260_230 = "purity260/230"
```

Field Value

[string](#)

PURITY260_280

```
public const string PURITY260_280 = "purity260/280"
```

Field Value

[string](#)

REFERENCE

```
public const string REFERENCE = "reference"
```

Field Value

[string](#)

RESULTS

```
public const string RESULTS = "results"
```

Field Value

[string](#) 

SAMPLE

```
public const string SAMPLE = "sample"
```

Field Value

[string](#) 

SELFTEST

```
public const string SELFTEST = "selftest"
```

Field Value

[string](#) 

SELFTEST_AMPLIFIERS

```
public const string SELFTEST_AMPLIFIERS = "amplifiers"
```

Field Value

[string](#) 

SELFTEST_AMPLIFIER_CURRENT

```
public const string SELFTEST_AMPLIFIER_CURRENT = "current"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_REFERENCE1

```
public const string SELFTEST_AMPLIFIER_REFERENCE1 = "reference1"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_REFERENCE11

```
public const string SELFTEST_AMPLIFIER_REFERENCE11 = "reference11"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_REFERENCE111

```
public const string SELFTEST_AMPLIFIER_REFERENCE111 = "reference111"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_SAMPLE1

```
public const string SELFTEST_AMPLIFIER_SAMPLE1 = "sample1"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_SAMPLE11

```
public const string SELFTEST_AMPLIFIER_SAMPLE11 = "sample11"
```

Field Value

[string](#)

SELFTEST_AMPLIFIER_SAMPLE111

```
public const string SELFTEST_AMPLIFIER_SAMPLE111 = "sample111"
```

Field Value

[string](#)

SELFTEST_DARKREFERENCE

```
public const string SELFTEST_DARKREFERENCE = "darkreference"
```

Field Value

[string](#)

SELFTEST_DARKSAMPLE

```
public const string SELFTEST_DARKSAMPLE = "darksample"
```

Field Value

[string](#) 

SELFTEST_ILED

```
public const string SELFTEST_ILED = "iled"
```

Field Value

[string](#) 

SELFTEST_REFERENCE

```
public const string SELFTEST_REFERENCE = "reference"
```

Field Value

[string](#) 

SELFTEST_RESULT

```
public const string SELFTEST_RESULT = "result"
```

Field Value

[string](#) 

SELFTEST_SAMPLE


```
public const string SELFTEST_SAMPLE = "sample"
```

Field Value

[string](#)

SELFTEST_SPLITRATION230NM

```
public const string SELFTEST_SPLITRATION230NM = "splitration230nm"
```

Field Value

[string](#)

SERIALNUMBER

```
public const string SERIALNUMBER = "serialnumber"
```

Field Value

[string](#)

VALUES

```
public const string VALUES = "values"
```

Field Value

[string](#)

dsDNA

```
public const string dsDNA = "dsDNA"
```

Field Value

[string](#) 

ssDNA

```
public const string ssDNA = "ssDNA"
```

Field Value

[string](#) 

ssRNA

```
public const string ssRNA = "ssRNA"
```

Field Value

[string](#) 

Enum Error

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing various error codes.

```
public enum Error
```

Fields

```
INVALID_PARAMETER = 2
```

```
LEVELLING_FAILED = 100
```

```
OK = 0
```

```
SREC_FLASH_WRITE_ERROR = 4
```

```
SREC_INVALID_CRC = 6
```

```
SREC_INVALID_STRING = 7
```

```
SREC_UNSUPPORTED_TYPE = 5
```

```
UNKNOWN_COMMAND = 1
```

Class Factors

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Correction factors due to different light absorption at different wavelengths of the cuvette.

```
public class Factors
```

Inheritance

[object](#)  ← Factors

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

Factors(Quadruple?, Quadruple?)

Initializes a new instance of the [Factors](#) class with optional correction factors.

```
public Factors(Quadruple? factorAirToBlank = null, Quadruple? factor340ToNNN = null)
```

Parameters

factorAirToBlank [Quadruple](#)

Correction factor for air to blank measurements.

factor340ToNNN [Quadruple](#)

Correction factor for F340 to NNN measurements.

Properties

AirToBlank

Gets or sets the correction factor for air to blank measurements.

```
public Quadruple AirToBlank { get; set; }
```

Property Value

[Quadruple](#)

F340ToNNN

Gets or sets the correction factor for F340 to NNN measurements.

```
public Quadruple F340ToNNN { get; set; }
```

Property Value

[Quadruple](#)

Methods

ToString()

Returns a string representation of the correction factors.

```
public override string ToString()
```

Returns

[string](#)

A formatted string displaying the correction factors.

Operators

operator +(Factors, Factors)

Defines the addition operator for [Factors](#).

```
public static Factors operator +(Factors left, Factors right)
```

Parameters

left [Factors](#)

The left operand.

right [Factors](#)

The right operand.

Returns

[Factors](#)

A new [Factors](#) instance with the sum of corresponding values.

operator /(Factors, Factors)

Defines the division operator for [Factors](#).

```
public static Factors operator /(Factors left, Factors right)
```

Parameters

left [Factors](#)

The left operand.

right [Factors](#)

The right operand.

Returns

[Factors](#)

A new [Factors](#) instance with the quotient of corresponding values.

operator /(Factors, double)

Defines the division operator for [Factors](#) with a scalar value.

```
public static Factors operator /(Factors left, double scalar)
```

Parameters

left [Factors](#)

The left operand.

scalar [double](#)

The scalar value.

Returns

[Factors](#)

A new [Factors](#) instance with each value divided by the scalar.

Enum Index

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing different index values for hardware configurations.

```
public enum Index
```

Fields

LAST_MEASUREMENT_COUNT = 10

LED230NM_MAX_CURRENT = 23

LED260NM_MAX_CURRENT = 33

LED280NM_MAX_CURRENT = 43

LED340NM_MAX_CURRENT = 53

LEVELLING_LED230_AMPLIFICATIONREFERENCE = 153

LEVELLING_LED230_AMPLIFICATIONSAMPLE = 152

LEVELLING_LED230_CURRENT = 151

LEVELLING_LED230_SETUPRESULT = 150

LEVELLING_LED260_AMPLIFICATIONREFERENCE = 163

LEVELLING_LED260_AMPLIFICATIONSAMPLE = 162

LEVELLING_LED260_CURRENT = 161

LEVELLING_LED260_SETUPRESULT = 160

LEVELLING_LED280_AMPLIFICATIONREFERENCE = 173

LEVELLING_LED280_AMPLIFICATIONSAMPLE = 172

LEVELLING_LED280_CURRENT = 171

LEVELLING_LED280_SETUPRESULT = 170

LEVELLING_LED340_AMPLIFICATIONREFERENCE = 183

LEVELLING_LED340_AMPLIFICATIONSAMPLE = 182

LEVELLING_LED340_CURRENT = 181

LEVELLING_LED340_SETUPRESULT = 180

PRODUCTIONNUMBER = 3

SELFTEST_AMPLIFIER_CURRENT = 101

SELFTEST_AMPLIFIER_REFERENCE1 = 105

SELFTEST_AMPLIFIER_REFERENCE11 = 106

SELFTEST_AMPLIFIER_REFERENCE111 = 107

SELFTEST_AMPLIFIER_SAMPLE1 = 102

SELFTEST_AMPLIFIER_SAMPLE11 = 103

SELFTEST_AMPLIFIER_SAMPLE111 = 104

SELFTEST_AMPLIFIER_SETUPRESULT = 108

SELFTEST_AMPLIFIER_SPLITRATIO230NM = 100

SELFTEST_LED230_DARKREFERENCE = 112

SELFTEST_LED230_DARKSAMPLE = 111

SELFTEST_LED230_ILED = 110

SELFTEST_LED230_REFERENCE = 114

SELFTEST_LED230_SAMPLE = 113

SELFTEST_LED260_DARKREFERENCE = 122

SELFTEST_LED260_DARKSAMPLE = 121

SELFTEST_LED260_ILED = 120

SELFTEST_LED260_REFERENCE = 124

SELFTEST_LED260_SAMPLE = 123

SELFTEST_LED280_DARKREFERENCE = 132

SELFTEST_LED280_DARKSAMPLE = 131

SELFTEST_LED280_ILED = 130

SELFTEST_LED280_REFERENCE = 134

SELFTEST_LED280_SAMPLE = 133

SELFTEST_LED340_DARKREFERENCE = 142

SELFTEST_LED340_DARKSAMPLE = 141

SELFTEST_LED340_ILED = 140

SELFTEST_LED340_REFERENCE = 144

SELFTEST_LED340_SAMPLE = 143

SERIALNUMBER = 1

VERSION = 0

Enum Led

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing different led wave lengths.

```
public enum Led
```

Fields

```
LED_230 = 1
```

```
LED_260 = 2
```

```
LED_280 = 3
```

```
LED_340 = 4
```

```
LED_OFF = 0
```

Class LevellingChannelResult

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents the result of a levelling operation for a single measurement channel.

```
public class LevellingChannelResult
```

Inheritance

[object](#)  ← LevellingChannelResult

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

LevellingChannelResult(int, int, int, int)

Initializes a new instance of the [LevellingChannelResult](#) class.

```
public LevellingChannelResult(int result, int current, int amplificationSample,  
int amplificationReference)
```

Parameters

result [int](#) 

The overall levelling result.

current [int](#) 

The applied current in uA during levelling.

amplificationSample [int](#) 

The amplification value for the sample.

amplificationReference [int](#) 

The amplification value for the reference.

Properties

AmplificationReference

Gets or sets the amplification value for the reference measurement.

```
public int AmplificationReference { get; set; }
```

Property Value

[int](#)

Remarks

0: factor 1.1 1: factor 11.0 2: factor 111.0

AmplificationSample

Gets or sets the amplification value for the sample measurement.

```
public int AmplificationSample { get; set; }
```

Property Value

[int](#)

Remarks

0: factor 1.1 1: factor 11.0 2: factor 111.0

Current

Gets or sets the current in uA applied during the levelling process.

```
public int Current { get; set; }
```

Property Value

[int](#)

Result

Gets or sets the overall result of the levelling operation.

```
public int Result { get; set; }
```

Property Value

[int](#)

Methods

ToJson()

Converts the levelling result into a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)

A [JsonNode](#) containing levelling result data.

ToString()

Returns a string representation of the levelling result.

```
public override string ToString()
```

Returns

[string](#)

A formatted string with levelling parameters.

Class LevellingResult

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents the overall levelling results for multiple measurement channels.

```
public class LevellingResult
```

Inheritance

[object](#)  ← LevellingResult

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

LevellingResult(LevellingChannelResult, LevellingChannelResult, LevellingChannelResult, LevellingChannelResult)

Initializes a new instance of the [LevellingResult](#) class.

```
public LevellingResult(LevellingChannelResult channel1230, LevellingChannelResult channel1260,  
    LevellingChannelResult channel1280, LevellingChannelResult channel1340)
```

Parameters

channel1230 [LevellingChannelResult](#)

Levelling result for the 230nm channel.

channel1260 [LevellingChannelResult](#)

Levelling result for the 260nm channel.

channel1280 [LevellingChannelResult](#)

Levelling result for the 280nm channel.

channel1340 [LevellingChannelResult](#)

Levelling result for the 340nm channel.

Properties

Channel230

Gets or sets the levelling result for the 230nm measurement channel.

```
public LevellingChannelResult Channel230 { get; set; }
```

Property Value

[LevellingChannelResult](#)

Channel260

Gets or sets the levelling result for the 260nm measurement channel.

```
public LevellingChannelResult Channel260 { get; set; }
```

Property Value

[LevellingChannelResult](#)

Channel280

Gets or sets the levelling result for the 280nm measurement channel.

```
public LevellingChannelResult Channel280 { get; set; }
```

Property Value

[LevellingChannelResult](#)

Channel340

Gets or sets the levelling result for the 340nm measurement channel.

```
public LevellingChannelResult Channel340 { get; set; }
```

Property Value

[LevellingChannelResult](#)

Methods

ToJson()

Converts the levelling results into a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)

A [JsonNode](#) containing levelling results for all channels.

ToString()

Returns a string representation of the levelling results for all channels.

```
public override string ToString()
```

Returns

[string](#)

A formatted string containing levelling results for all channels.

Class Measurement

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents a measurement containing baseline, air, and sample values, along with optional comments.

```
public class Measurement
```

Inheritance

[object](#)  ← Measurement

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

Measurement(SingleMeasurement, SingleMeasurement, SingleMeasurement, string)

Initializes a new instance of the [Measurement](#) class.

```
public Measurement(SingleMeasurement baseline, SingleMeasurement air, SingleMeasurement  
sample, string comment = "")
```

Parameters

baseline [SingleMeasurement](#)

The baseline measurement.

air [SingleMeasurement](#)

The air measurement.

sample [SingleMeasurement](#)

The sample measurement.

comment [string](#)

An optional comment for the measurement.

Methods

Absorbance(Quadruple?)

Calculates the absorbance based on air and sample measurements.

```
public Quadruple Absorbance(Quadruple? airToBlank = null)
```

Parameters

airToBlank [Quadruple](#)

Optional air-to-blank correction factor.

Returns

[Quadruple](#)

The absorbance value as a [Quadruple](#).

CalculateAbsorbance(SingleMeasurement, SingleMeasurement, Quadruple?)

Calculates the absorbance based on baseline and sample measurements.

```
public static Quadruple CalculateAbsorbance(SingleMeasurement baseline, SingleMeasurement measurement, Quadruple? correctionFactor = null)
```

Parameters

baseline [SingleMeasurement](#)

The baseline measurement.

measurement [SingleMeasurement](#)

The sample measurement.

`correctionFactor` [Quadruple](#)

An optional correction factor for absorbance calculation.

Returns

[Quadruple](#)

The absorbance values as a [Quadruple](#).

Comment()

Gets the comment associated with the measurement.

```
public string Comment()
```

Returns

[string](#) 

The comment text.

Factors()

Computes the correction factors for the measurement. To get valid factors, the sample must not contain DNA or RNA (i.e. buffer only),

```
public Factors Factors()
```

Returns

[Factors](#)

A [Factors\(\)](#) object containing computed correction factors.

FromJson(JsonNode?)

Creates a [Measurement](#) instance from a JSON representation.

```
public static Measurement FromJson(JsonNode? node)
```

Parameters

node [JsonNode](#)[↗]

The JSON node containing the measurement data.

Returns

[Measurement](#)

A [Measurement](#) object populated from JSON data.

Exceptions

[ArgumentNullException](#)[↗]

Thrown if the provided JSON node is null.

[InvalidOperationException](#)[↗]

Thrown if required fields are missing.

Results(Factors)

Computes the final results using given correction factors.

```
public Results Results(Factors factors)
```

Parameters

factors [Factors](#)

The correction factors.

Returns

[Results](#)

A [Results\(Factors\)](#) object containing calculated values.

SetComment(string)

Sets the comment for the measurement.

```
public void SetComment(string comment)
```

Parameters

comment [string](#)

The comment text.

ToJson()

Converts the measurement to a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)

A JSON node representing the measurement.

ToString()

Returns a string representation of the measurement.

```
public override string ToString()
```

Returns

[string](#)

A formatted string displaying baseline, air, and sample measurements.

Class Quadruple

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents a quadruple set of measurement values at different wavelengths. This class performs mathematical operations (plus, minus, ...) on all four wavelengths at the same time.

```
public class Quadruple
```

Inheritance

[object](#)  ← Quadruple

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Constructors

Quadruple(double, double?, double?, double?)

Initializes a new instance of the [Quadruple](#) class.

```
public Quadruple(double value230 = 0, double? value260 = null, double? value280 = null, double? value340 = null)
```

Parameters

value230 [double](#) 

Value at 230 nm.

value260 [double](#) ?

Value at 260 nm (default: same as value230).

value280 [double](#) ?

Value at 280 nm (default: same as value230).

value340 [double](#)?

Value at 340 nm (default: same as value230).

Properties

Value230

Gets or sets the value at 230 nm.

```
public double Value230 { get; set; }
```

Property Value

[double](#)

Value260

Gets or sets the value at 260 nm.

```
public double Value260 { get; set; }
```

Property Value

[double](#)

Value280

Gets or sets the value at 280 nm.

```
public double Value280 { get; set; }
```

Property Value

[double](#)

Value340

Gets or sets the value at 340 nm.

```
public double Value340 { get; set; }
```

Property Value

[double](#)

Methods

Abs()

Returns a new instance with absolute values.

```
public Quadruple Abs()
```

Returns

[Quadruple](#)

ToString()

Returns a string representation of the quadruple values.

```
public override string ToString()
```

Returns

[string](#)

Operators

operator +(Quadruple, object)

Adds a quadruple or scalar value to this instance.

```
public static Quadruple operator +(Quadruple lhs, object rhs)
```

Parameters

lhs [Quadruple](#)

rhs [object](#)

Returns

[Quadruple](#)

operator /(Quadruple, object)

Divides this instance by a quadruple or scalar value.

```
public static Quadruple operator /(Quadruple lhs, object rhs)
```

Parameters

lhs [Quadruple](#)

rhs [object](#)

Returns

[Quadruple](#)

operator >=(Quadruple, Quadruple)

Determines whether this instance is greater than or equal to another quadruple.

```
public static bool operator >=(Quadruple lhs, Quadruple rhs)
```

Parameters

lhs [Quadruple](#)

rhs [Quadruple](#)

Returns

[bool](#)

operator <=(Quadruple, Quadruple)

Determines whether this instance is less than or equal to another quadruple.

```
public static bool operator <=(Quadruple lhs, Quadruple rhs)
```

Parameters

lhs [Quadruple](#)

rhs [Quadruple](#)

Returns

[bool](#)

operator *(Quadruple, object)

Multiplies this instance by a quadruple or scalar value.

```
public static Quadruple operator *(Quadruple lhs, object rhs)
```

Parameters

lhs [Quadruple](#)

rhs [object](#)

Returns

[Quadruple](#)

operator -(Quadruple, object)

Subtracts a quadruple or scalar value from this instance.

```
public static Quadruple operator -(Quadruple lhs, object rhs)
```

Parameters

lhs [Quadruple](#)

rhs [object](#)

Returns

[Quadruple](#)

Class Results

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents the results of a nucleic acid analysis, including DNA and RNA concentrations and purity ratios.

```
public class Results
```

Inheritance

[object](#)  ← Results

Inherited Members

[object.Equals\(object, object\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ReferenceEquals\(object, object\)](#) 

Constructors

Results(double, double, double, double, double)

Initializes a new instance of the [Results](#) class with specified nucleic acid concentrations and purity ratios.

```
public Results(double dsDNA, double ssDNA, double ssRNA, double purityRatio260_230,  
double purityRatio260_280)
```

Parameters

dsDNA [double](#) 

Double-stranded DNA concentration.

ssDNA [double](#) 

Single-stranded DNA concentration.

ssRNA [double](#) 

Single-stranded RNA concentration.

`purityRatio260_230` [double](#)

260/230 purity ratio.

`purityRatio260_280` [double](#)

260/280 purity ratio.

Properties

DsDNA

Gets or sets the concentration of double-stranded DNA (dsDNA) in ng/ul.

```
public double DsDNA { get; set; }
```

Property Value

[double](#)

PurityRatio260_230

Gets or sets the 260/230 purity ratio, which indicates potential contamination by organic compounds or chaotropic salts.

```
public double PurityRatio260_230 { get; set; }
```

Property Value

[double](#)

PurityRatio260_280

Gets or sets the 260/280 purity ratio, which indicates protein contamination.

```
public double PurityRatio260_280 { get; set; }
```


Property Value

[double](#)

SsDNA

Gets or sets the concentration of single-stranded DNA (ssDNA) in ng/ul.

```
public double SsDNA { get; set; }
```

Property Value

[double](#)

SsRNA

Gets or sets the concentration of single-stranded RNA (ssRNA) in ng/ul.

```
public double SsRNA { get; set; }
```

Property Value

[double](#)

Methods

Equals(object?)

Determines whether the specified object is equal to the current instance.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current instance.

Returns

[bool](#)[↗]

`true` if the objects are equal; otherwise, `false`.

FromJson(JsonNode?)

Creates a [Results](#) instance from a JSON representation.

```
public static Results FromJson(JsonNode? node)
```

Parameters

`node` [JsonNode](#)[↗]

The JSON node containing the nucleic acid data.

Returns

[Results](#)

A [Results](#) object populated from the JSON data.

Exceptions

[ArgumentNullException](#)[↗]

Thrown if the provided JSON node is null.

[InvalidOperationException](#)[↗]

Thrown if required fields are missing or null.

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

[int](#)

A hash code for the current object.

ToJson()

Converts the current instance to a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)

A [JsonNode](#) representing the current object.

ToString()

Returns a string representation of the results.

```
public override string ToString()
```

Returns

[string](#)

A formatted string with dsDNA, ssDNA, ssRNA, and purity ratios.

Class SelfTestResult

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents the result of a self-test, containing various checks and problem indicators.

```
public class SelfTestResult
```

Inheritance

[object](#)  ← SelfTestResult

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

SelfTestResult(int)

Initializes a new instance of the [SelfTestResult](#) class.

```
public SelfTestResult(int result)
```

Parameters

result [int](#) 

The result of the self-test.

Properties

Result

Gets or sets the integer result of the self-test.

```
public int Result { get; set; }
```

Property Value

[int](#)

Methods

HasProblemWithILed230()

Checks if there is a problem with the current flow of the 230nm LED.

```
public bool HasProblemWithILed230()
```

Returns

[bool](#)

HasProblemWithILed260()

Checks if there is a problem with the current flow of the 260nm LED.

```
public bool HasProblemWithILed260()
```

Returns

[bool](#)

HasProblemWithILed280()

Checks if there is a problem with the current flow of the 280nm LED.

```
public bool HasProblemWithILed280()
```

Returns

[bool](#)

HasProblemWithILed340()

Checks if there is a problem with the current flow of the 340nm LED.

```
public bool HasProblemWithILed340()
```

Returns

[bool](#)

HasProblemWithLevelling230()

Checks if there is a problem with levelling at 230 nm.

```
public bool HasProblemWithLevelling230()
```

Returns

[bool](#)

HasProblemWithReferencDetector()

Checks if there is a problem with the reference detector.

```
public bool HasProblemWithReferencDetector()
```

Returns

[bool](#)

HasProblemWithReference230()

Checks whether there is a problem with the reference detector when the 230nm LED lights up.

```
public bool HasProblemWithReference230()
```

Returns

[bool](#)

HasProblemWithReference260()

Checks whether there is a problem with the reference detector when the 260nm LED lights up.

```
public bool HasProblemWithReference260()
```

Returns

[bool](#)

HasProblemWithReference280()

Checks whether there is a problem with the reference detector when the 280nm LED lights up.

```
public bool HasProblemWithReference280()
```

Returns

[bool](#)

HasProblemWithReference340()

Checks whether there is a problem with the reference detector when the 340nm LED lights up.

```
public bool HasProblemWithReference340()
```

Returns

[bool](#)

HasProblemWithSample230()

Checks whether there is a problem with the sample detector when the 230nm LED lights up.

```
public bool HasProblemWithSample230()
```

Returns

[bool](#)

HasProblemWithSample260()

Checks whether there is a problem with the sample detector when the 260nm LED lights up.

```
public bool HasProblemWithSample260()
```

Returns

[bool](#)

HasProblemWithSample280()

Checks whether there is a problem with the sample detector when the 280nm LED lights up.

```
public bool HasProblemWithSample280()
```

Returns

[bool](#)

HasProblemWithSample340()

Checks whether there is a problem with the sample detector when the 340nm LED lights up.

```
public bool HasProblemWithSample340()
```


Returns

[bool](#)

HasProblemWithSampleDetector()

Checks if there is a problem with the sample detector.

```
public bool HasProblemWithSampleDetector()
```

Returns

[bool](#)

HasProblemWithSplitration()

Checks if there is a problem with the split ratio between sample and reference detector.

```
public bool HasProblemWithSplitration()
```

Returns

[bool](#)

HasProblems()

Determines whether there are any problems in the self-test.

```
public bool HasProblems()
```

Returns

[bool](#)

True if there are problems; otherwise, false.

ToJson()

Converts the self-test result to a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)

A JsonNode representing the self-test result.

Enum Selftest

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Flags enumeration for self-test components.

[Flags]

```
public enum Selftest
```

Fields

ILED_230 = 1

ILED_260 = 2

ILED_280 = 4

ILED_340 = 8

LEVELLING_230 = 32768

REFERENCE = 4096

REFERENCE_230 = 256

REFERENCE_260 = 512

REFERENCE_280 = 1024

REFERENCE_340 = 2048

SAMPLE = 8192

SAMPLE_230 = 16

SAMPLE_260 = 32

SAMPLE_280 = 64

SAMPLE_340 = 128

SPLITRATIO = 16384

Class SingleMeasurement

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Represents a single measurement consisting of multiple wavelength channels.

```
public class SingleMeasurement
```

Inheritance

[object](#)  ← SingleMeasurement

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) 

Constructors

SingleMeasurement(Channel?, Channel?, Channel?, Channel?)

Initializes a new instance of the [SingleMeasurement](#) class with optional channels.

```
public SingleMeasurement(Channel? channel1230 = null, Channel? channel1260 = null, Channel?  
channel1280 = null, Channel? channel1340 = null)
```

Parameters

channel1230 [Channel](#)

The channel for 230 nm (default: new Channel()).

channel1260 [Channel](#)

The channel for 260 nm (default: new Channel()).

channel1280 [Channel](#)

The channel for 280 nm (default: new Channel()).

channel1340 [Channel](#)

The channel for 340 nm (default: new Channel()).

Properties

Channel230

Gets or sets the channel for 230 nm measurements.

```
public Channel Channel230 { get; set; }
```

Property Value

[Channel](#)

Channel260

Gets or sets the channel for 260 nm measurements.

```
public Channel Channel260 { get; set; }
```

Property Value

[Channel](#)

Channel280

Gets or sets the channel for 280 nm measurements.

```
public Channel Channel280 { get; set; }
```

Property Value

[Channel](#)

Channel340

Gets or sets the channel for 340 nm measurements.

```
public Channel Channel340 { get; set; }
```

Property Value

[Channel](#)

Methods

FromJson(JsonNode)

Creates a [SingleMeasurement](#) instance from a JSON node.

```
public static SingleMeasurement FromJson(JsonNode node)
```

Parameters

node [JsonNode](#)[↗]

The JSON node containing measurement data.

Returns

[SingleMeasurement](#)

A new [SingleMeasurement](#) instance.

Exceptions

[ArgumentNullException](#)[↗]

Thrown if the JSON node is null or missing required properties.

Reference()

Returns a quadruple representing the reference values from all channels.

```
public Quadruple Reference()
```

Returns

[Quadruple](#)

A [Quadruple](#) containing the reference values.

Sample()

Returns a quadruple representing the sample values from all channels.

```
public Quadruple Sample()
```

Returns

[Quadruple](#)

A [Quadruple](#) containing the sample values.

ToJson()

Converts the measurement data to a JSON representation.

```
public JsonNode ToJson()
```

Returns

[JsonNode](#)[↗](#)

A JsonNode representing the measurement.

ToString()

Returns a string representation of the measurement values.

```
public override string ToString()
```

Returns

[string](#) 

A formatted string containing all channel values.

Enum StatusLedColor

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing different status led colors .

```
public enum StatusLedColor
```

Fields

BLUE = 3

GREEN = 2

OFF = 0

RED = 1

Class StorageMeasurement

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Handles storage and retrieval of measurements using JSON serialization.

```
public class StorageMeasurement
```

Inheritance

[object](#)  ← StorageMeasurement

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

StorageMeasurement(string)

Initializes a new instance of the [StorageMeasurement](#) class, optionally loading from a file.

```
public StorageMeasurement(string filename = "")
```

Parameters

filename [string](#) 

Optional filename to load measurements from.

Properties

Count

Gets the number of stored measurements.

```
public int Count { get; }
```

Property Value

[int](#)

this[int]

```
public Measurement this[int index] { get; }
```

Parameters

index [int](#)

Property Value

[Measurement](#)

Methods

Append(Measurement, string)

Appends a measurement to storage.

```
public void Append(Measurement measurement, string comment = "")
```

Parameters

measurement [Measurement](#)

The measurement to append.

comment [string](#)

An optional comment for the measurement.

AppendWithResults(Measurement, Results, string)

Appends a measurement and the results to storage.

```
public void AppendWithResults(Measurement measurement, Results results, string comment = "")
```

Parameters

measurement [Measurement](#)

The measurement to append.

results [Results](#)

The results to append.

comment [string](#) 

An optional comment for the measurement.

Measurements()

Retrieves the list of stored measurements.

```
public List<Measurement> Measurements()
```

Returns

[List](#)  <[Measurement](#)>

A list of [Measurement](#) objects.

Results()

Retrieves the list of stored results.

```
public List<Results> Results()
```

Returns

[List](#) <[Results](#)>

A list of [Measurement](#) objects.

Save(string)

Saves the measurement data to a specified file.

```
public void Save(string filename)
```

Parameters

filename [string](#)

The filename to save data to.

Enum TypeOf

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Enumeration representing different data types.

```
public enum TypeOf
```

Fields

```
DOUBLE = 2
```

```
STRING = 0
```

```
UINT32 = 1
```

Class USB

Namespace: [Hse.EviDense](#)

Assembly: EviDense.dll

Provides USB-related constants.

```
public static class USB
```

Inheritance

[object](#)  ← USB

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

PID

Product ID (PID) of the USB device.

```
public const int PID = 2
```

Field Value

[int](#) 

VID

Vendor ID (VID) of the USB device.

```
public const int VID = 7358
```

Field Value

